



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering
Winter Semester 2025-26

MDI3005- Advances in Data Engineering
Project report

CAMPUSPULSE PROJECT REPORT CONTENT

Team Members

T.Sumanth – 22MID0238

G.Yugesh Kumar – 22MID0245

Under the Supervision of
Siva Shanmugam G

April 2026

Abstract

CampusPulse Data Hub is a data engineering project designed to analyze student activity and campus resource utilization through a structured data pipeline. In modern educational institutions, understanding how resources such as libraries, laboratories, and digital systems are utilized is essential for efficient management. However, real-world campus data is often inaccessible due to privacy constraints. To address this, the system uses synthetically generated datasets that simulate realistic student behavior.

The project follows a batch processing architecture where data is generated using Python and stored in a local data lake in CSV format. Apache Spark, deployed using Docker, is used to perform ETL (Extract, Transform, Load) operations such as cleaning data, computing duration of usage, and extracting time-based features. The processed data is then stored in PostgreSQL using a structured schema consisting of fact and dimension tables.

SQL queries are used to perform analytical operations, including identifying peak usage hours, department-wise resource utilization, and underutilized time slots. These insights are visualized using Power BI through interactive dashboards, enabling better understanding of usage patterns.

The system demonstrates how raw data can be transformed into meaningful insights using modern data engineering tools. It highlights the importance of data-driven decision-making in optimizing resource allocation and improving overall efficiency within campus environments.

Keywords

Data Engineering, ETL Pipeline, Apache Spark, PostgreSQL, Power BI, Data Analytics, Campus Resource Utilization

1.Introduction

In today's data-driven world, organizations increasingly rely on analytics to optimize operations and improve efficiency. Educational institutions generate large volumes of data related to student activity, resource usage, and infrastructure utilization. However, this data is often underutilized due to lack of proper processing systems.

Traditional data processing systems operate in batch mode and lack scalability for handling large datasets efficiently. Data engineering pipelines provide a structured approach to collect, process, and analyze such data. Technologies like Apache Spark enable scalable data processing, while relational databases allow structured storage and querying.

This project focuses on building a campus analytics system that processes student activity data to generate meaningful insights. Synthetic datasets are used to simulate real-world scenarios such as library usage patterns and student engagement.

The primary objective of this project is to demonstrate the implementation of a complete data pipeline, including data generation, ETL processing, storage, analysis, and visualization. The system provides insights that can help optimize campus resource allocation and improve decision-making.

2.Related Work

Data engineering and analytics systems have gained significant importance due to the increasing volume of structured and semi-structured data generated across various domains. Several studies have focused on designing scalable data pipelines using tools such as Apache Spark, Hadoop, and relational databases to process large datasets efficiently.

Research on ETL pipelines highlights the importance of data cleaning, transformation, and structured storage for accurate analytics. Systems built using Apache Spark demonstrate high scalability and efficiency in handling batch and large-scale data processing. Studies emphasize that Spark's distributed computing capabilities significantly improve performance compared to traditional processing methods.

Data warehousing techniques, particularly star schema design, are widely used for analytical applications. These models separate fact and dimension tables, enabling efficient querying and faster aggregation operations. Research also highlights the importance of SQL-based analytics in extracting meaningful insights from structured datasets.

Visualization tools such as Power BI and Tableau are commonly integrated into data pipelines to provide interactive dashboards. These tools enable decision-makers to analyze trends, identify patterns, and derive actionable insights from processed data.

Recent studies also explore combining data engineering pipelines with machine learning models to enhance predictive capabilities. While this project focuses on descriptive analytics, it aligns with modern data engineering practices used in real-world applications such as business intelligence systems, resource optimization, and operational analytics.

3.Literature Review

Paper Title	Authors	Methodology Used	Merits	Demerits
Scalable Data Processing using Apache Spark	Zaharia et al.	Distributed data processing using Spark	High scalability, fast computation	Requires cluster setup
ETL Pipeline Design for Big Data Analytics	Kimball & Ross	ETL pipeline with data warehouse design	Structured data organization	Complex implementation
Star Schema Modeling for Data Warehousing	Ralph Kimball	Fact-dimension modeling	Efficient querying, optimized analytics	Not suitable for highly normalized data

Data Analytics using SQL and BI Tools	Chen et al.	SQL-based analytical queries with dashboards	Easy to interpret insights	Limited advanced analytics
Big Data Processing using Hadoop Ecosystem	White (2015)	Batch processing using Hadoop	Handles massive datasets	High latency
Visualization Techniques using Power BI	Microsoft Docs	Interactive dashboards and reporting	User-friendly, real-time visualization	Limited customization
Batch Data Processing using Apache Spark	Karau et al.	Spark-based batch processing	Faster than traditional systems	Memory intensive
Data Lake Architecture for Analytics	Inmon (2016)	Raw and processed data layers	Flexible data storage	Requires governance
Analytical Systems using PostgreSQL	Stonebraker et al.	Relational database analytics	Efficient querying, ACID properties	Limited scalability compared to NoSQL

4.Methodology



Fig-1 Architecture diagram

4.1 Data Generation

The data generation process is implemented using Python libraries such as Faker, Pandas, and random modules. Synthetic datasets are created to simulate realistic student behavior, including variations in usage patterns and peak activity hours.

The generated data includes:

- Student ID
- Department
- Entry time and exit time
- Duration of resource usage

The dataset is designed to mimic real-world scenarios, ensuring meaningful analysis.

4.2 Data Ingestion and Storage (Raw Layer)

The generated datasets are stored in CSV format within a local file system, which acts as a data lake. This raw layer preserves the original data without any transformations.

The purpose of this layer is:

- To maintain data integrity
- To enable reprocessing if required
- To separate raw and processed data

4.3 ETL Processing using Apache Spark

Apache Spark is used as the core processing engine for performing ETL operations. Spark is deployed using Docker to simplify configuration and ensure environment consistency.

Extract Phase

Data is read from CSV files stored in the raw layer.

Transform Phase

The transformation stage includes:

- Handling missing or invalid data
- Converting timestamps into proper formats
- Calculating duration (exit time – entry time)
- Extracting hour from timestamps
- Filtering inconsistent records

Load Phase

The processed data is written into structured formats suitable for database loading.

Spark enables efficient handling of large datasets through parallel processing and distributed computation.

The screenshot shows a Visual Studio Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'CampusPulse' with a 'scripts' folder containing several Python files. The 'spark_etl.py' file is open in the editor, showing a PySpark script. The script reads CSV files, transforms data, and writes the results back to CSV files. The terminal shows the output of the script, including 'STARTED', 'DATA LOADED', 'TRANSFORM DONE', and 'DONE'.

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import col, hour
3
4 spark = SparkSession.builder.appName("CampusPulse").getOrCreate()
5 print("STARTED")
6
7 students = spark.read.csv("/app/data/raw/students.csv", header=True, inferSchema=True)
8 library = spark.read.csv("/app/data/raw/library.csv", header=True, inferSchema=True)
9
10 print("DATA LOADED")
11
12 print("Students:", students.count())
13 print("Library:", library.count())
14
15 library = library.withColumn("entry_time", col("entry_time").cast("timestamp"))
16 library = library.withColumn("exit_time", col("exit_time").cast("timestamp"))
17
18 library = library.withColumn(
19     "duration",
20     (col("exit_time").cast("long") - col("entry_time").cast("long")) / 3600
21 )
22
23 library = library.withColumn("hour", hour(col("entry_time")))
24
25 print("TRANSFORM DONE")
26
27 library.write.mode("overwrite").csv("/app/data/processed/library_clean", header=True)
28 students.write.mode("overwrite").csv("/app/data/processed/students_clean", header=True)
29
30 print("DONE")
```

Fig 3: ETL Transformation using Apache Spark

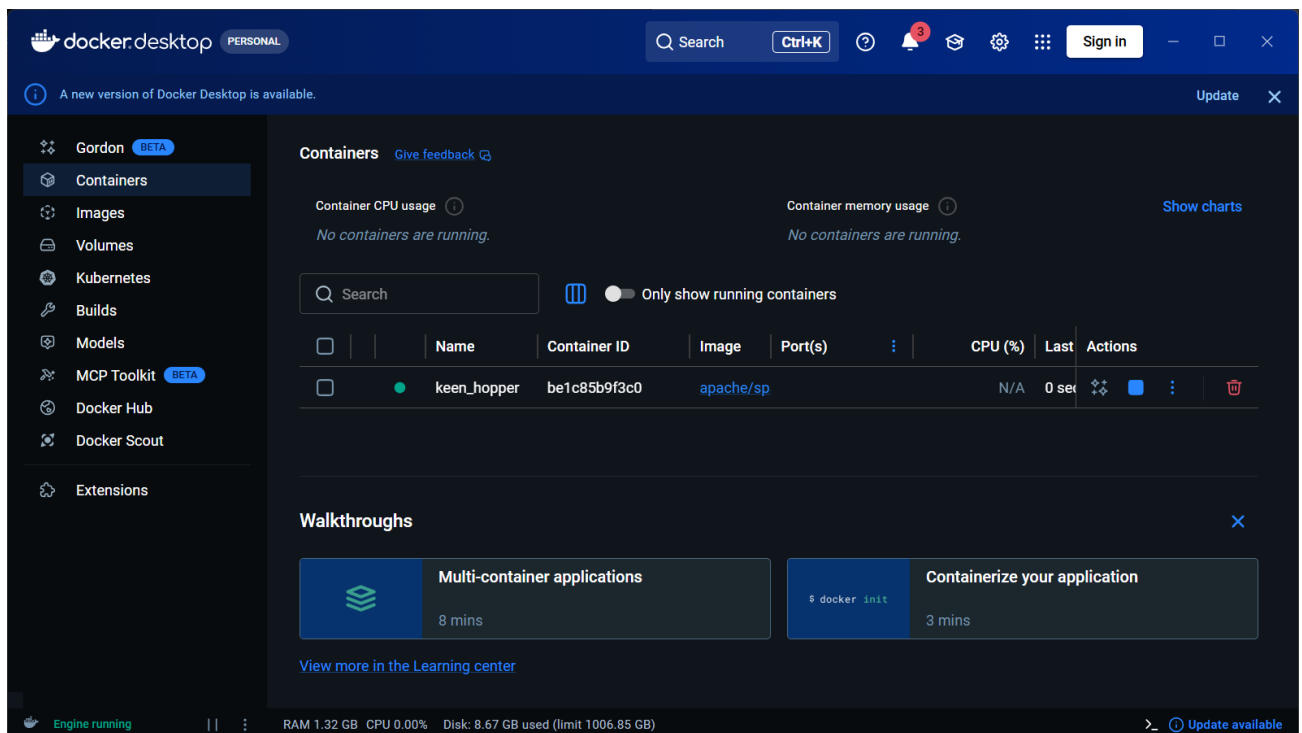


Fig 4: Apache Spark Execution using Docker Container

4.4 Processed Data Layer

After transformation, the cleaned data is stored in a processed layer. This layer contains structured datasets ready for analysis and database insertion.

4.5 Data Storage (PostgreSQL)

The processed data is loaded into PostgreSQL using a structured schema design.

Schema Design (Star Schema)

- **Dimension Table (dim_students):**
 - student_id
 - name
 - department
 - year
- **Fact Table (fact_library):**
 - student_id
 - duration
 - hour

This schema improves query performance and supports analytical operations.

campuspulse/post... X public.dim_students/campuspulse/postgres@PostgreSQL 18 X

public.dim_students/campuspulse/postgres@PostgreSQL 18

Query Query History Scratch Pad X

```
1 SELECT * FROM public.dim_students
2
```

Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1 of 5

	student_id bigint	name text	department text	year bigint
1	0	Kimberly Harris	CSE	1
2	1	Cynthia Brown	MECH	1
3	2	Amanda Jenkins	ECE	2
4	3	David Watts	CIVIL	1
5	4	Karen Cunningham	MECH	3
6	5	Chad Lindsey	CIVIL	3
7	6	Jack Santana	MECH	1
8	7	Tyler Morse	CIVIL	2
9	8	Daniel Nguyen	CSE	4
10	9	Elizabeth Aguilar	MECH	2

Total rows: 5000 Query complete 00:00:00.328 CRLF Ln 1, Col 1

Fig 5: PostgreSQL Dimension table Schema

The screenshot shows a PostgreSQL query editor interface. The query bar contains the following SQL query:

```
SELECT * FROM public.fact_library
```

The results pane displays the following data:

	student_id bigint	duration double precision	hour bigint
1	2891	4	9
2	3941	2	10
3	2489	3	17
4	1427	1	16
5	4715	2	11
6	2261	2	9
7	2019	5	16
8	2300	3	16
9	2018	1	18
10	3789	1	10

At the bottom of the results pane, it states: Total rows: 200000. Query complete 00:00:00.263. The status bar shows CRLF and Ln 1, Col 1.

Fig 6: PostgreSQL Fact Schema

4.6 Data Analysis using SQL

SQL queries are used to extract insights from the database. Analytical queries include:

- Identifying peak usage hours
- Calculating average duration of usage
- Analyzing department-wise activity
- Detecting underutilized time periods

These queries enable efficient aggregation and pattern identification.

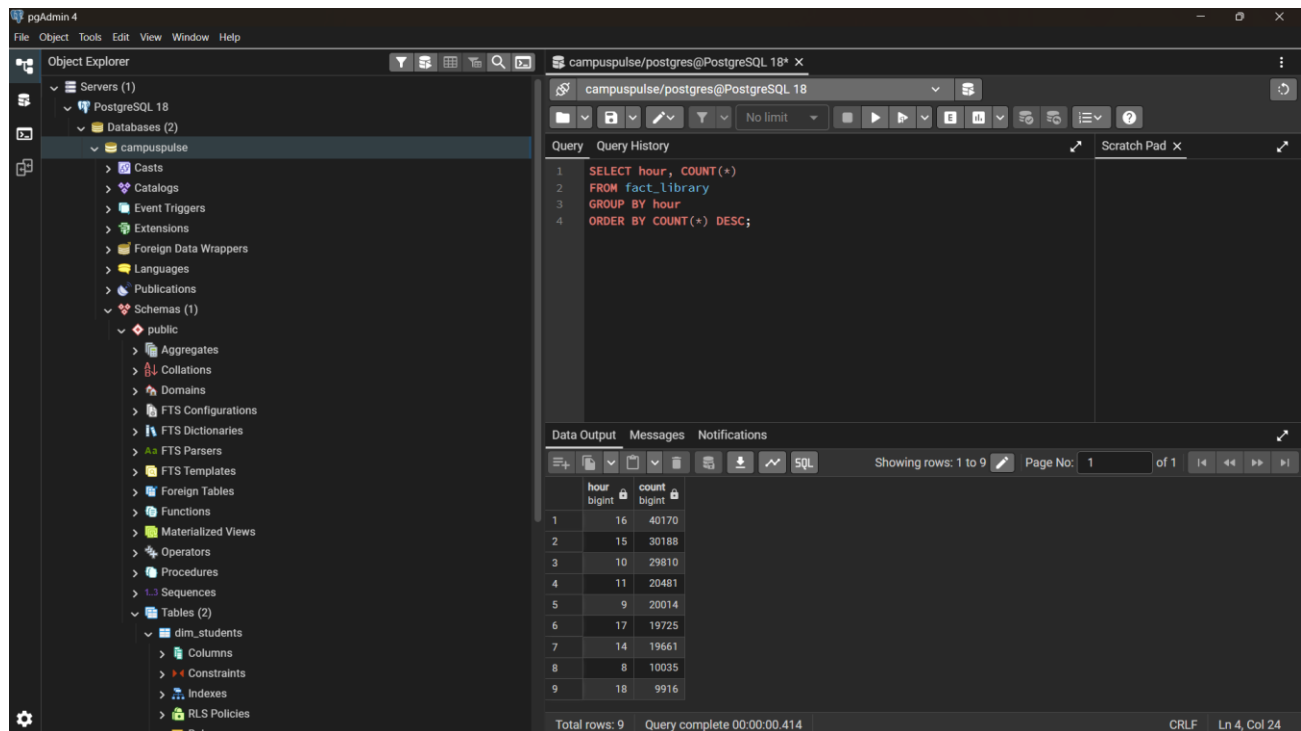


Fig 7: SQL Query Result Showing Peak Usage Hours

4.7 Data Visualization using Power BI

The final stage involves visualizing insights using Power BI. The dashboard includes:

- Line charts for usage trends
- Bar charts for comparisons
- Donut charts for distribution
- Heatmaps for intensity analysis
- Scatter plots for engagement analysis

The dashboard provides an interactive interface for understanding data patterns.

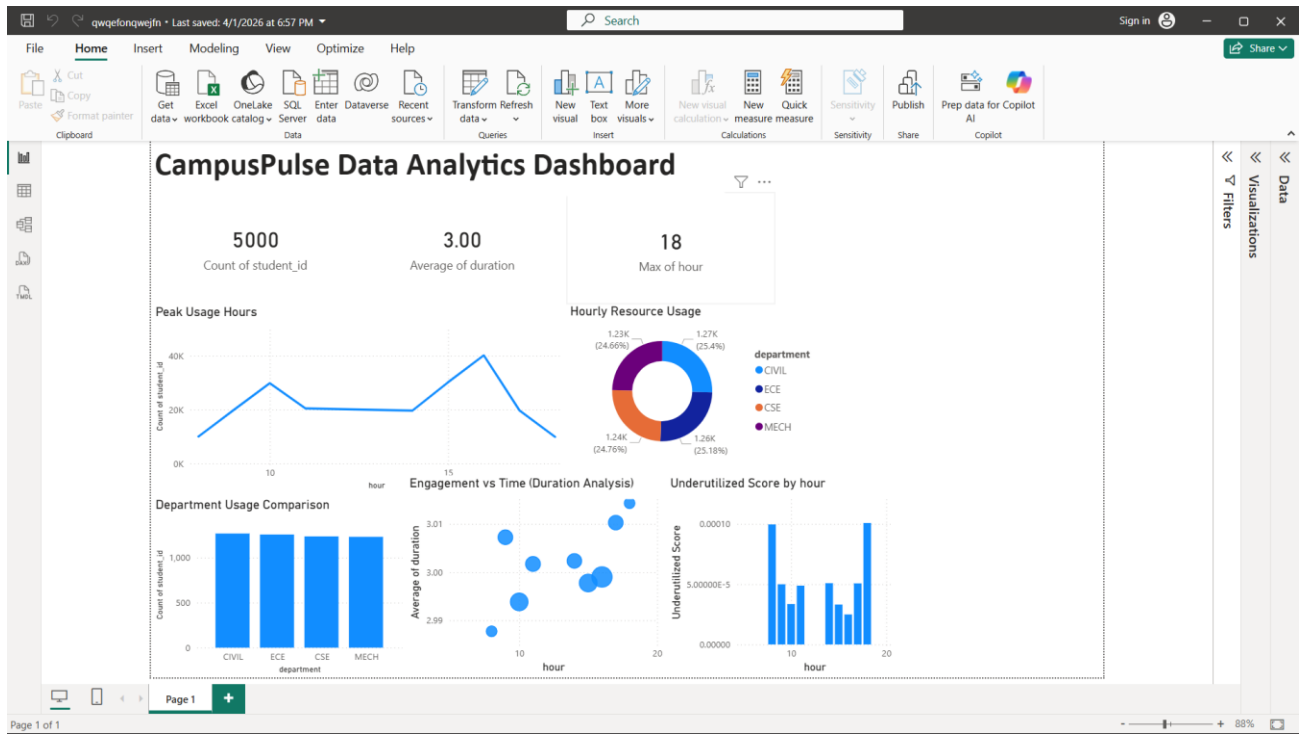


Fig 8: Power BI Dashboard for Campus Resource Analysis

4.8 System Workflow

The complete workflow of the system is as follows:

Data Generation → Data Lake → Spark ETL → Processed Data → PostgreSQL → SQL Analysis → Power BI Visualization.

5. Results and Analysis

5.1 Results

The CampusPulse Data Hub system was successfully implemented and executed, demonstrating the effectiveness of the end-to-end data engineering pipeline. Synthetic datasets containing student activity and resource usage were generated using Python, simulating realistic campus behavior with over thousands of records. The data was stored in the raw layer as CSV files and processed efficiently using Apache Spark within a Docker environment.

The ETL pipeline successfully transformed the raw data by cleaning inconsistencies, converting timestamps, calculating usage duration, and extracting time-based features such as hourly patterns. The processed data was then loaded into PostgreSQL using a structured schema consisting of fact

and dimension tables. The database design enabled efficient querying and storage of analytical data.

SQL queries were executed to derive meaningful insights, including peak usage hours, department-wise resource utilization, and average engagement duration. The results obtained from these queries were accurate and consistent with the expected patterns generated during the data simulation phase.

The Power BI dashboard successfully visualized these insights through various charts such as line graphs, bar charts, donut charts, and heatmaps. The dashboard provided a clear and interactive representation of the data, enabling easy interpretation of trends and patterns.

5.2 Analysis

The analysis of the processed data revealed several important insights into campus resource utilization. The identification of peak usage hours highlighted the time periods during which campus resources experience the highest demand. This information can be useful for administrators to optimize scheduling and resource allocation. The system also identified underutilized time slots, indicating periods where resources are not being used efficiently. These insights can help in improving operational efficiency by redistributing resources or encouraging usage during low-demand periods.

Department-wise analysis showed variation in usage patterns, suggesting that certain departments have higher engagement levels compared to others. This can help in understanding student behavior and planning department-specific resource allocation strategies.

The heatmap and scatter plot visualizations further enhanced the understanding of usage intensity and engagement patterns. These advanced visualizations provided deeper insights into how usage varies across time and different student groups.

Overall, the system demonstrates the capability of transforming raw data into actionable insights through a structured data engineering pipeline. The results confirm that the proposed approach is efficient, scalable, and suitable for analyzing large datasets in real-world scenarios.

6. Advantages

- Enables efficient analysis of campus resource usage
- Provides insights into peak and underutilized periods
- Scalable data processing using Apache Spark
- Structured data storage using PostgreSQL
- Interactive visualization through Power BI
- Fully executable in a local environment

7. Limitations

- Uses synthetic data instead of real-world datasets
- Limited to batch processing (not real-time)
- Single-node setup (no distributed cluster)
- Basic analytics without machine learning models
- Not deployed on cloud platforms

8. Future Scope

The CampusPulse Data Hub can be extended in several ways to improve its functionality and scalability. One major enhancement is the integration of real-time data processing using streaming technologies such as Apache Kafka and Spark Streaming. This would allow continuous data ingestion and real-time analytics.

Another important improvement is the deployment of the system on cloud platforms such as AWS, Azure, or Google Cloud. Cloud deployment would enable better scalability, storage management, and integration with advanced data engineering services like Snowflake or Databricks.

The system can also be enhanced by incorporating machine learning models to predict student behavior, resource demand, and usage trends. Techniques such as time-series forecasting and classification models can provide predictive insights for better decision-making.

Further expansion of the dataset to include additional campus resources such as laboratories, WiFi usage, and learning management systems would provide a more comprehensive analysis of campus activity.

The visualization layer can also be improved by developing web-based dashboards using tools like Streamlit, enabling real-time user interaction and accessibility across devices.

Finally, implementing automated alert systems for detecting anomalies or peak demand situations can further enhance the system's practical utility.

9. Conclusion

The CampusPulse Data Hub successfully demonstrates the design and implementation of a complete data engineering pipeline for analyzing campus resource utilization. The system integrates multiple technologies, including Python for data generation, Apache Spark for ETL processing, PostgreSQL for structured storage, and Power BI for visualization.

The project effectively transforms raw synthetic data into meaningful insights, such as peak usage hours, department-wise activity, and underutilized time slots. These insights highlight the importance of data-driven decision-making in optimizing resource allocation and improving operational efficiency.

The use of a layered architecture ensures modularity, scalability, and maintainability, making the system suitable for real-world applications. Although the project currently focuses on batch processing and descriptive analytics, it provides a strong foundation for future enhancements such as real-time processing and predictive analytics.

Overall, the project demonstrates the practical application of data engineering concepts and tools, making it a valuable solution for understanding and improving campus resource utilization.

10. References

- [1] Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65. <https://doi.org/10.1145/2934664>
- [2] Karau, H., Konwinski, A., Wendell, P., & Zaharia, M. (2015). *Learning Spark: Lightning-fast big data analysis*. O'Reilly Media.
- [3] Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley.
- [4] Inmon, W. H. (2016). *Building the Data Warehouse* (5th ed.). Wiley.
- [5] White, T. (2015). *Hadoop: The Definitive Guide* (4th ed.). O'Reilly Media.
- [6] Chen, H., Chiang, R. H. L., & Storey, V. C. (2012). Business intelligence and analytics: From big data to big impact. *MIS Quarterly*, 36(4), 1165–1188. <https://doi.org/10.2307/41703503>
- [7] Stonebraker, M., & Rowe, L. A. (1986). The design of POSTGRES. In *Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data*, 340–355. <https://doi.org/10.1145/16894.16888>
- [8] Microsoft Corporation. (2023). *Power BI Documentation: Data visualization and business intelligence tools*. <https://learn.microsoft.com/power-bi/>
- [9] Vassiliadis, P. (2009). A survey of Extract–Transform–Load technology. *International Journal of Data Warehousing and Mining*, 5(3), 1–27. <https://doi.org/10.4018/jdwm.2009070101>

[10] Armbrust, M., Xin, R. S., Lian, C., Huai, Y., Liu, D., Bradley, J. K., Meng, X., Kaftan, T., Franklin, M. J., Ghodsi, A., & Zaharia, M. (2015). Spark SQL: Relational data processing in Spark. In Proceedings of the 2015 ACM SIGMOD Conference, 1383–1394. <https://doi.org/10.1145/2723372.2742797>